

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Sensor dan Aktuator

Muhammad Arifin Umasangadji - 5024241083

2026

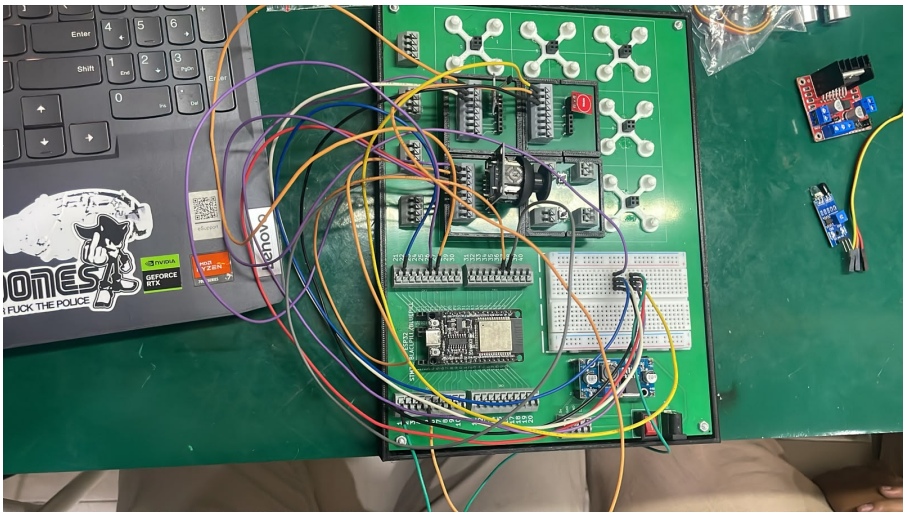
1 Langkah-Langkah Percobaan

1.1 Persiapan Alat dan Bahan

Sebelum dimulai, perlu menyiapkan beberapa alat dan bahan yang akan digunakan, yaitu modul board praktikum dengan ESP32 DevKit, adapter 12V 3A, kabel USB, joystick KY-023, touch sensor TTP223, *hall effect sensor* beserta magnet, sensor ultrasonik HC-SR04, sensor IR *obstacle*, motor DC *brushed* dengan driver L298N, motor stepper 28BYJ-48 dengan driver ULN2003, motor servo, motor *brushless* (BLDC) dengan ESC, obeng, serta kabel jumper male-male dan male-female. Laptop juga perlu mensetup Visual Studio Code dan ekstensi PlatformIO.

1.2 Percobaan 1: Sensor Resistif, Kapasitif, dan Induktif

1. Merangkai joystick KY-023, touch sensor TTP223, dan *hall effect sensor* ke ESP32. Pin VRx, VRy, dan SW joystick dihubungkan ke GPIO 34, 35, dan 27, pin OUT TTP223 ke GPIO 4, dan pin OUT *hall effect sensor* ke GPIO 26 dengan VCC dan GND sesuai skema pada modul.



Gambar 1: Rangkaian Percobaan 1

2. Menuliskan program pembacaan ketiga sensor pada file `src/main.cpp`, kemudian melakukan Build dan Upload ke ESP32.

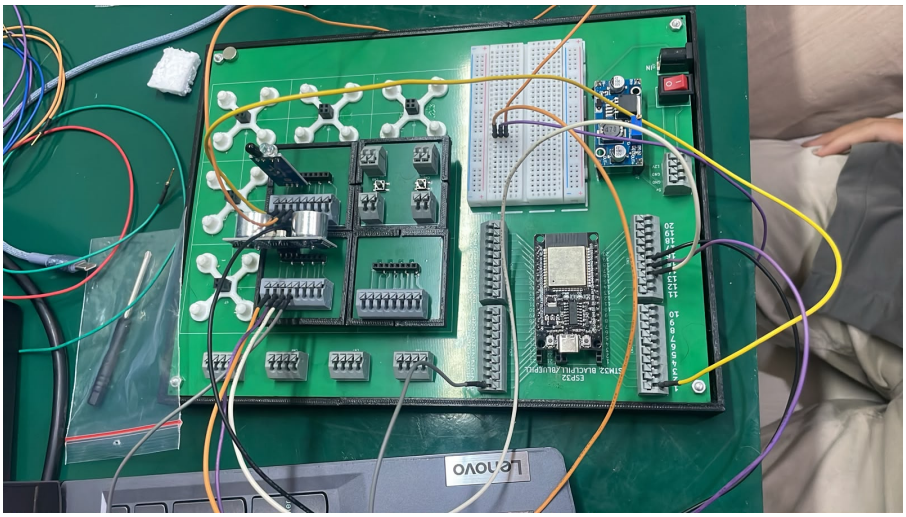
3. Membuka Serial Monitor dengan baud rate 115200, lalu menggerakkan joystick, menyentuh pad TTP223, dan mendekatkan magnet ke *hall effect sensor* untuk mengamati perubahan nilai masing-masing sensor.

```
Executing task in folder modul2-sensor-aktuator: C:\Users\ASUS\.platformio\penv\Scripts\platformi
o.exe device monitor
Joystick VRx: 2288 | VRy: 4095 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 2197 | VRy: 4095 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 2131 | VRy: 4095 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 1865 | VRy: 3344 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 1866 | VRy: 1909 | SW: idle | Touch (TTP223): TERSENTUH | Hall Effect: idle
Joystick VRx: 1872 | VRy: 1894 | SW: idle | Touch (TTP223): TERSENTUH | Hall Effect: idle
Joystick VRx: 1872 | VRy: 1904 | SW: idle | Touch (TTP223): TERSENTUH | Hall Effect: idle
Joystick VRx: 1867 | VRy: 1904 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 1871 | VRy: 0 | SW: idle | Touch (TTP223): TERSENTUH | Hall Effect: idle
Joystick VRx: 1872 | VRy: 0 | SW: idle | Touch (TTP223): TERSENTUH | Hall Effect: idle
Joystick VRx: 1902 | VRy: 0 | SW: idle | Touch (TTP223): TERSENTUH | Hall Effect: idle
Joystick VRx: 1872 | VRy: 0 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 1872 | VRy: 1905 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 1872 | VRy: 1906 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 1872 | VRy: 0 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 1871 | VRy: 0 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 1872 | VRy: 1903 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 1872 | VRy: 0 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 1872 | VRy: 0 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 1870 | VRy: 1906 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 1870 | VRy: 0 | SW: idle | Touch (TTP223): idle | Hall Effect: MAGNET TERDETEKSI
Joystick VRx: 1872 | VRy: 1899 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 1872 | VRy: 1905 | SW: idle | Touch (TTP223): idle | Hall Effect: MAGNET TERDETEKSI
Joystick VRx: 1872 | VRy: 1903 | SW: idle | Touch (TTP223): idle | Hall Effect: MAGNET TERDETEKSI
Joystick VRx: 1873 | VRy: 0 | SW: idle | Touch (TTP223): idle | Hall Effect: MAGNET TERDETEKSI
Joystick VRx: 2096 | VRy: 0 | SW: idle | Touch (TTP223): idle | Hall Effect: MAGNET TERDETEKSI
Joystick VRx: 1867 | VRy: 1889 | SW: idle | Touch (TTP223): idle | Hall Effect: MAGNET TERDETEKSI
Joystick VRx: 1869 | VRy: 1905 | SW: idle | Touch (TTP223): idle | Hall Effect: MAGNET TERDETEKSI
Joystick VRx: 1872 | VRy: 1907 | SW: idle | Touch (TTP223): TERSENTUH | Hall Effect: MAGNET TERDETEKS
```

Gambar 2: Hasil pembacaan sensor pada Serial Monitor

1.3 Percobaan 2: Sensor Ultrasonik dan IR Obstacle

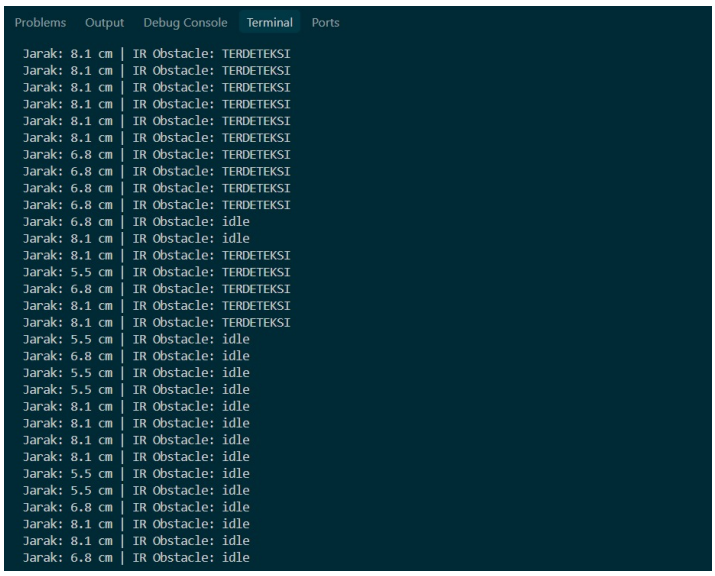
1. Merangkai sensor ultrasonik HC-SR04 dengan pin Trig ke GPIO 5 dan pin Echo ke GPIO 18, serta sensor IR *obstacle* dengan pin OUT ke GPIO 19.



Gambar 3: Rangkaian Percobaan 2

2. Menuliskan program pembacaan jarak dan status sensor IR pada `src/main.cpp`, kemudian melakukan Build dan Upload.

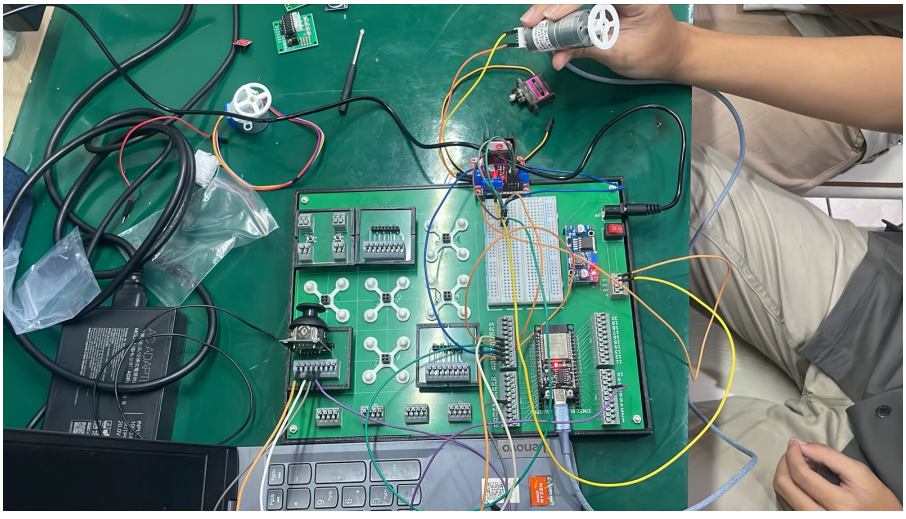
3. Membuka Serial Monitor, lalu mendekatkan dan menjauhkan objek di depan kedua sensor untuk mengamati nilai jarak (cm) dan status deteksi sensor IR.



Gambar 4: Hasil pembacaan jarak dan sensor IR pada Serial Monitor

1.4 Percobaan 3: Aktuator Motor DC

1. Merangkai motor DC, driver L298N (jumper ENA dilepas), adaptor 12V, dan joystick ke ESP32. Pin VRx dihubungkan ke GPIO 34, SW ke GPIO 27, ENA ke GPIO 25, IN1 ke GPIO 26, dan IN2 ke GPIO 33. GND adaptor 12V, L298N, dan ESP32 disatukan.



Gambar 5: Rangkaian Percobaan 3

2. Menuliskan program kontrol motor DC menggunakan PWM, kemudian melakukan Build dan Upload.

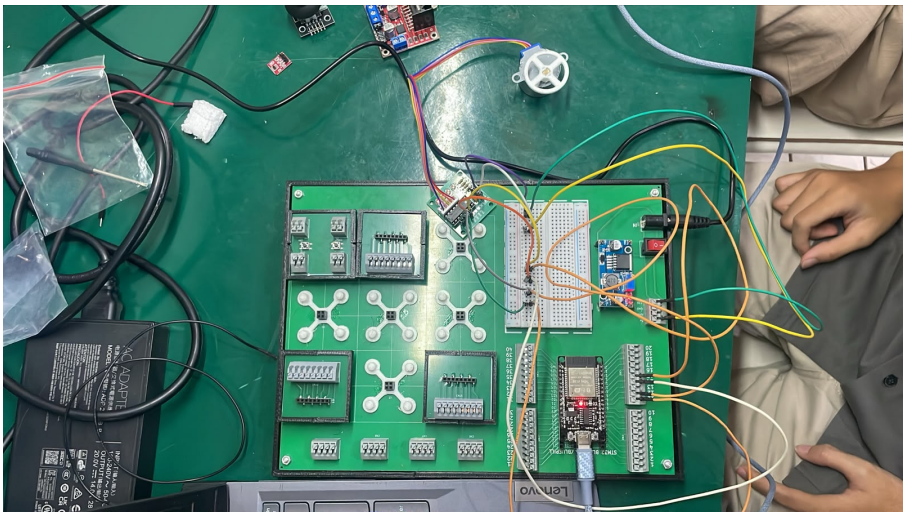
3. Membuka Serial Monitor, lalu menggeser joystick ke kiri dan kanan serta menekan tombol SW untuk mengamati arah putar, kecepatan motor, dan status yang ditampilkan.

```
Motor: MUNDUR, PWM = 83
Motor: MUNDUR, PWM = 81
Motor: MUNDUR, PWM = 80
Motor: idle (joystick netral)
Motor: MUNDUR, PWM = 78
Motor: idle (joystick netral)
Motor: MUNDUR, PWM = 77
Motor: MUNDUR, PWM = 89
Motor: MUNDUR, PWM = 92
Motor: MUNDUR, PWM = 83
Motor: MAJU, PWM = 53
Motor: MUNDUR, PWM = 85
Motor: MUNDUR, PWM = 93
Motor: MUNDUR, PWM = 84
Motor: MUNDUR, PWM = 80
```

Gambar 6: Hasil kontrol motor DC pada Serial Monitor

1.5 Percobaan 4: Aktuator Motor Stepper

1. Merangkai motor stepper 28BYJ-48 pada driver ULN2003. Pin IN1 sampai IN4 driver dihubungkan ke GPIO 16, 17, 18, dan 19, dengan catu daya eksternal dan GND yang disatukan dengan ESP32.



Gambar 7: Rangkaian Percobaan 4

2. Menuliskan program yang memutar motor sejauh 90° (1024 step) dengan jeda 0,5 detik, kemudian melakukan Build dan Upload.

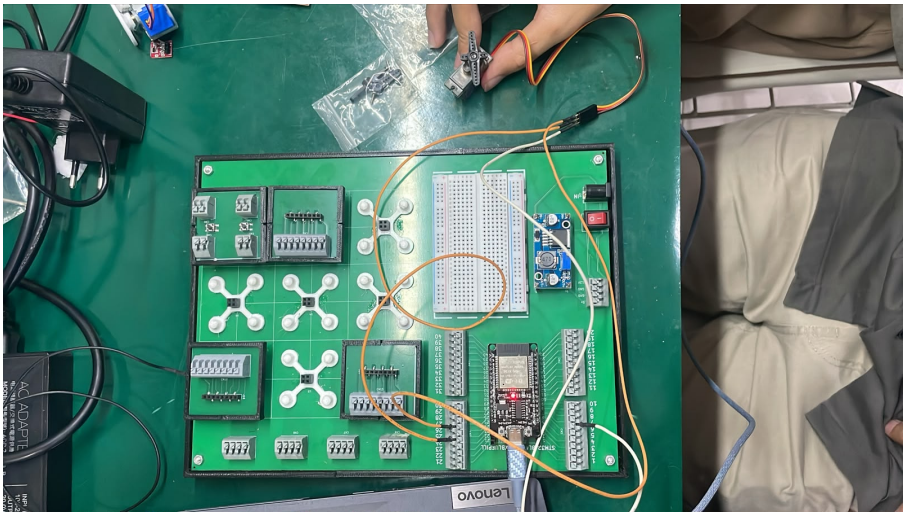
3. Membuka Serial Monitor dan mengamati poros motor stepper yang berputar 90° lalu berhenti 0,5 detik secara berulang.

```
*  Executing task in folder modul2-sensor-aktuator: C:\Users\ASUS\.platformio\penv\Scripts\platformi
o.exe device monitor
Motor berputar 90 derajat
Motor berhenti 0.5 detik
Motor berputar 90 derajat
Motor berhenti 0.5 detik
Motor berputar 90 derajat
Motor berhenti 0.5 detik
Motor berputar 90 derajat
Motor berhenti 0.5 detik
Motor berputar 90 derajat
Motor berhenti 0.5 detik
Motor berputar 90 derajat
Motor berhenti 0.5 detik
Motor berputar 90 derajat
```

Gambar 8: Hasil program motor stepper pada Serial Monitor

1.6 Percobaan 5: Aktuator Motor Servo

1. Menambahkan library ESP32Servo pada `platformio.ini`, kemudian merangkai motor servo ke ESP32 dengan pin sinyal pada GPIO 13, VCC pada pin 5V, dan GND pada GND ESP32.



Gambar 9: Rangkaian Percobaan 5

2. Menuliskan program yang menggerakkan servo bolak-balik antara 180° dan 0°, kemudian melakukan Build dan Upload.

3. Membuka Serial Monitor dan mengamati lengan servo yang bergerak bolak-balik dengan jeda 0,5 detik sesuai status yang ditampilkan.

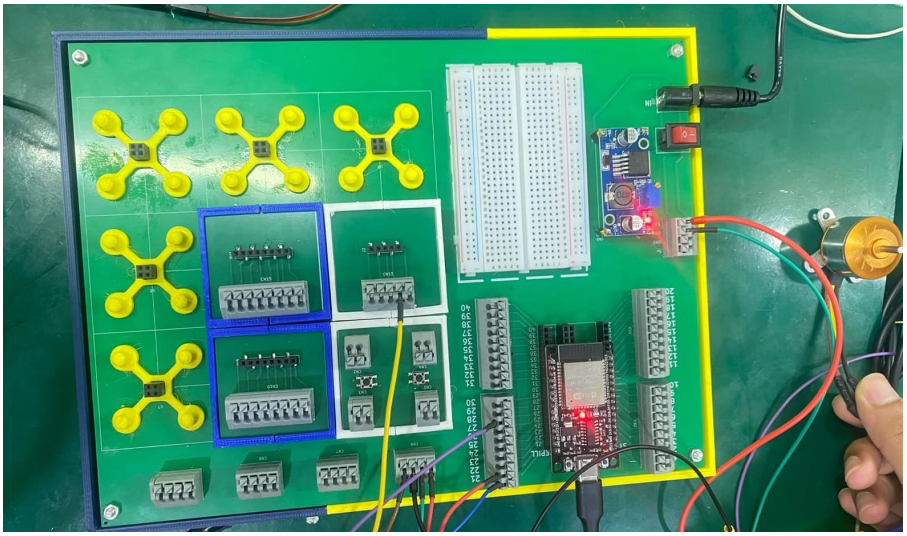
```
Problems  Output  Debug Console  Terminal  Ports

*  Executing task in folder modul2-sensor-aktuator: C:\Users\ASUS\.platformio\penv\Scripts\platformi
o.exe device monitor
Servo bergerak CCW ke 0 derajat
Gerakan selesai (estimasi)
Berhenti selama 0.5 detik
Servo bergerak CW ke 180 derajat
Gerakan selesai (estimasi)
Berhenti selama 0.5 detik
Servo bergerak CCW ke 0 derajat
Gerakan selesai (estimasi)
Berhenti selama 0.5 detik
Servo bergerak CW ke 180 derajat
Gerakan selesai (estimasi)
Berhenti selama 0.5 detik
Servo bergerak CCW ke 0 derajat
```

Gambar 10: Hasil program motor servo pada Serial Monitor

1.7 Percobaan 6: Aktuator ESC dan Motor Brushless (BLDC)

1. Merangkai ESC dan motor BLDC ke ESP32. Pin sinyal ESC dihubungkan ke GPIO 25, kabel 5V dan GND ESC ke pin VIN dan GND ESP32, daya utama ESC dari power eksternal 12V pada modul (saklar dalam posisi mati), dan tiga kabel fasa ESC ke motor BLDC. Propeller tidak dipasang pada motor.



Gambar 11: rangkaian percobaan 6

2. Menuliskan program arming ESC dan kontrol kecepatan menggunakan LEDC pada `src/main.cpp`, kemudian melakukan Build dan Upload.
3. Mencabut kabel USB dari ESP32, memastikan pin VIN dan GND ESP32 terpasang pada ESC, kemudian menyalakan power eksternal dari modul.
4. Mengamati perubahan kecepatan putar motor BLDC saat nilai PWM naik dari 0 ke 128 dan turun kembali dari 128 ke 0.

2 Analisis Hasil Percobaan

Pada percobaan pertama, joystick KY-023, touch sensor TTP223, dan *hall effect sensor* digunakan untuk membaca input melalui GPIO ESP32. Nilai joystick saat stick dilepas berada di sekitar 1870 sampai 1900, sedangkan nilai maksimal dan minimal VRy mencapai 4095 dan 0. Hal ini menunjukkan bahwa titik tengah joystick tidak selalu tepat di 2048 sehingga diperlukan *dead zone*. Sementara itu, TTP223 dan *hall effect sensor* berhasil mendeteksi sentuhan dan keberadaan magnet. Pada percobaan kedua, sensor HC-SR04 dan IR *obstacle* digunakan untuk mendeteksi objek. HC-SR04 mengukur jarak berdasarkan pantulan gelombang ultrasonik, sedangkan sensor IR menggunakan prinsip optik. Hasil pengukuran HC-SR04 menunjukkan jarak sekitar 5,5 sampai 8,1 cm.

Pada percobaan ketiga, motor DC dikendalikan menggunakan driver L298N dengan sinyal PWM berdasarkan posisi joystick. Motor dapat bergerak maju, mundur, dan berhenti sesuai nilai input yang diterima. Kecepatan motor dipengaruhi oleh besar nilai PWM, sedangkan kondisi berhenti terjadi ketika joystick berada dalam area *dead zone*. Pada percobaan keempat dan kelima, motor stepper 28BYJ-48 berhasil bergerak sejauh 90° secara berulang dengan jeda 0,5 detik. Motor servo juga

berhasil bergerak bolak-balik antara posisi 0° dan 180°. Kedua motor memiliki prinsip pengendalian yang berbeda, yaitu motor stepper berdasarkan jumlah langkah, sedangkan motor servo berdasarkan lebar pulsa.

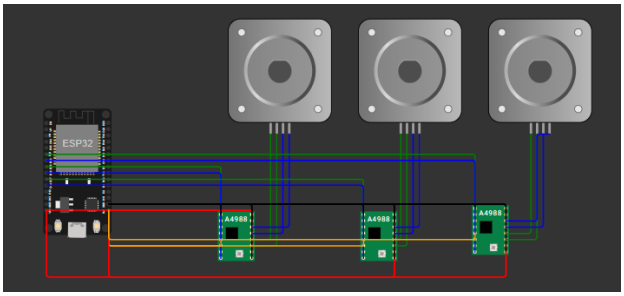
Pada percobaan keenam, ESC dikendalikan melalui sinyal PWM menggunakan GPIO 25 dengan frekuensi 50 Hz. Proses *arming* dilakukan dengan memberikan sinyal throttle minimum selama 5 detik, kemudian nilai PWM dinaikkan secara bertahap hingga sekitar 128 dan diturunkan kembali. Percobaan ini menunjukkan bahwa ESC dapat mengatur kerja motor berdasarkan sinyal throttle yang diberikan. Secara keseluruhan, keenam percobaan memperlihatkan penggunaan berbagai sensor sebagai perangkat masukan dan motor sebagai perangkat keluaran yang dikendalikan oleh ESP32.

3 Hasil Tugas Modul

Pembuatan simulasi jam analog dengan 3 motor stepper pada ESP32 menggunakan WOKWI. Motor pertama menunjukkan milidetik, motor kedua menunjukkan detik, dan motor ketiga menunjukkan menit. Setiap motor memakai driver A4988 dengan pin STEP dan DIR, yaitu GPIO 25 dan 26 (milidetik), GPIO 27 dan 14 (detik), serta GPIO 32 dan 33 (menit) :



Gambar 12: Hasil simulasi pada WOKWI



Gambar 13: Rangkaian Pada WOKWI

Berikut kode yang digunakan :

```
#include <Arduino.h>
#include <esp_timer.h>

const int stepPins[3] = {25, 27, 32};
const int dirPins[3] = {26, 14, 33};

const uint64_t stepsPerRev = 200;
const uint64_t periodUs[3] = {1000000ULL, 60000000ULL, 3600000000ULL};
const uint64_t speedUp = 1;

uint64_t stepsDone[3] = {0, 0, 0};
uint64_t startTime;
unsigned long lastPrint = 0;

```

```
15  void stepOnce(int i) {
16    digitalWrite(stepPins[i], HIGH);
17    delayMicroseconds(5);
18    digitalWrite(stepPins[i], LOW);
19    delayMicroseconds(5);
20  }
21
22  void setup() {
23    Serial.begin(115200);
24
25    for (int i = 0; i < 3; i++) {
26      pinMode(stepPins[i], OUTPUT);
27      pinMode(dirPins[i], OUTPUT);
28      digitalWrite(dirPins[i], HIGH);
29      digitalWrite(stepPins[i], LOW);
30    }
31
32    Serial.println("Jam analog 3 motor stepper");
33    startTime = esp_timer_get_time();
34  }
35
36  void loop() {
37    uint64_t elapsed = (esp_timer_get_time() - startTime) * speedUp;
38
39    for (int i = 0; i < 3; i++) {
40      uint64_t target = elapsed * stepsPerRev / periodUs[i];
41      while (stepsDone[i] < target) {
42        stepOnce(i);
43        stepsDone[i]++;
44      }
45    }
46
47    if (millis() - lastPrint >= 1000) {
48      lastPrint = millis();
49      uint64_t ms = elapsed / 1000;
50      Serial.printf("%02d:%02d.%03d\n", (int)((ms / 60000) % 60), (int)((ms / 1000) %
    60), (int)(ms % 1000));
51    }
52  }
```

Jawaban Pertanyaan Analisis :

1. Setiap jarum dibuat berputar satu putaran penuh (360°) dalam satu periode, yaitu 1 detik untuk milidetik, 60 detik untuk detik, dan 60 menit untuk menit. Motor stepper pada simulasi memiliki 1,8° per step sehingga satu putaran membutuhkan 200 step. Kecepatan putar dihitung dengan RPM = 60/T (T dalam detik), dan jeda antar step = T/200.

Jarum	Periode	RPM	Step/putaran	Jeda antar step
Milidetik	1 detik	60	200	5 ms
Detik	60 detik	1	200	300 ms
Menit	60 menit	0,0167	200	18 detik

Pada program, jumlah step yang seharusnya sudah dijalankan dihitung dari waktu yang berlalu, sehingga ketiga jarum tetap sinkron. Jika memakai 28BYJ-48 (4096 step), jeda antar step

menjadi lebih kecil, dan jarum milidetik (60 RPM) terlalu cepat untuk motor ini karena pada Percobaan 4 satu putaran penuh dengan jeda 2000 μs sudah memerlukan sekitar 8 detik.

2. Motor stepper pada simulasi tidak dapat diganti dengan motor servo. Servo standar hanya berputar pada rentang 0° sampai 180° sehingga jarum tidak dapat berputar penuh 360° secara terus-menerus seperti jam, dan setelah mencapai 180° harus kembali ke 0°. Servo juga dikendalikan berdasarkan sudut tujuan melalui lebar pulsa, bukan kecepatan putar yang diatur melalui jeda step seperti pada soal. Servo *continuous rotation* memang dapat berputar terus, tetapi hanya dapat diatur kecepatannya dan tidak memiliki kontrol posisi, sehingga jarum tidak akan presisi.

3. Driver berfungsi sebagai penguat arus dan penghubung antara ESP32 dengan motor. Pin GPIO ESP32 hanya dapat mengeluarkan arus puluhan mA pada 3,3V, sedangkan motor membutuhkan arus dan tegangan yang lebih besar. Driver juga mengatur hal yang tidak dapat dilakukan GPIO secara langsung, yaitu urutan pengaktifan koil pada stepper (ULN2003 dan A4988), pembalikan arah putar pada motor DC (L298N), dan komutasi tiga fasa pada motor BLDC (ESC), serta memisahkan catu daya motor dari ESP32 agar ESP32 terlindungi dari lonjakan arus dan tegangan balik. Karena itu setiap motor pada percobaan memakai driver. Pada servo, rangkaian driver sudah terpasang di dalam servo.

4 Kesimpulan

Praktikum ini berhasil dilakukan dan hasilnya sesuai dengan teori. Praktikan dapat membaca sensor resistif (joystick), kapasitif (TTP223), induktif (*hall effect*), ultrasonik, dan IR *obstacle* pada ESP32, serta mengendalikan motor DC dengan PWM, motor stepper berdasarkan jumlah step, motor servo berdasarkan sudut, dan motor BLDC melalui ESC dengan proses *arming*. Pada tugas modul, tiga motor stepper disimulasikan sebagai jarum milidetik, detik, dan menit dengan mengatur jeda step masing-masing. Setiap motor memerlukan driver karena pin ESP32 tidak mampu menyuplai arus motor secara langsung.

5 Lampiran

5.1 Dokumentasi saat praktikum

